

Bitácora Metodológica de Hallazgos

Frank Daza

17 de agosto de 2026

Contenido

1 Propósito y convenciones	4
1.1 Flujo documental	4
1.2 Trazabilidad obligatoria	4
1.3 Política de bibliografía	5
1.4 Plantilla estándar de hallazgo	5
1.5 Mapeo de fragmentos por fase	5
2 Fase 0 — Fundamentos reproducibles y datos	6
2.0.1 Task-1: Infraestructura y entorno reproducible	6
Objetivo.	6
Procedimiento.	6
Resultados.	6
Interpretación.	6
Decisiones.	6
Limitaciones.	6
2.0.2 Task-2: Estructura LaTeX del trabajo de grado y bitácora de hallazgos	7
Objetivo.	7
Procedimiento.	7
Resultados.	7
Interpretación.	7
Decisiones.	7
Limitaciones.	7

2.0.3	Task-3: Auditoría del Brain Tumor MRI Dataset (A4)	8
	Objetivo.	8
	Procedimiento.	8
	Resultados.	8
	Interpretación.	9
	Decisiones.	9
	Limitaciones.	10
2.0.4	Task-4: Contrato de métricas, logging y artefactos	10
	Objetivo.	10
	Procedimiento.	10
	Resultados.	10
	Interpretación.	11
	Decisiones.	11
	Limitaciones.	11
2.0.5	Task-5: Pipeline de preprocesamiento y aumento de datos (A5)	12
	Objetivo.	12
	Procedimiento.	12
	Resultados.	12
	Interpretación.	13
	Decisiones.	13
	Limitaciones.	13
2.0.6	Task-6: Protocolo de particiones — holdout, k-fold estratificado y fracciones (A7)	14
	Objetivo.	14
	Procedimiento.	14
	Decisión D1 frente a la redacción literal de A8.	14
	Resultados.	15
	Interpretación.	16
	Decisiones.	16
	Limitaciones.	16
3	Fase 1 — Arquitectura híbrida CNN-VQC	16
3.0.1	Task-7: Extractor congelado y fábrica de backbones (A1)	16
	Objetivo.	16
	Procedimiento.	17
	Resultados.	17
	Interpretación.	17
	Decisiones.	17
	Limitaciones.	18
3.0.2	Task-8: Bucle unificado de entrenamiento y evaluación (Trainer)	18
	Objetivo.	18
	Procedimiento.	18
	Resultados.	19

	Interpretación.	19
	Decisiones.	19
	Limitaciones.	19
3.0.3	Task-9: Clasificador cuántico variacional de 4 qubits (A2)	20
	Objetivo.	20
	Procedimiento.	20
	Resultados.	20
	Interpretación.	22
	Decisiones.	23
	Limitaciones.	23
3.0.4	Task-10: Modelo híbrido HQCNN con TorchLayer (A3)	23
	Objetivo.	23
	Procedimiento.	23
	Resultados.	24
	Interpretación.	25
	Decisiones.	25
	Limitaciones.	25
3.0.5	Task-11: Ablación de profundidad L y presupuesto de cómputo (A2)	26
	Objetivo.	26
	Protocolo (decisión de diseño, no A8).	26
	Criterio de selección (pre-declarado).	26
	Procedimiento.	26
	Resultados.	27
	Interpretación CE vs. F1.	27
	Presupuesto extrapolado.	27
	Decisiones.	28
	Limitaciones.	28
4	Fases 2 y 3 — Experimentación y análisis	29
4.0.1	Task-12: Líneas base clásicas con el 100 % del conjunto (A6)	29
	Objetivo.	29
	Procedimiento.	29
	Resultados por fold (validación, fracción 1.00).	29
	Agregación sobre 5 folds (media $\pm$ desv. estándar).	30
	Contraste con la literatura.	30
	Interpretación.	30
	Decisiones.	31
	Limitaciones.	31
	Addenda (decisión D3): reejecución en CUDA.	31
4.0.2	Task-13: Campaña experimental k-fold en escenarios de escasez (A8)	31
	Objetivo.	31
	Diseño e implementación.	32
	Estado de ejecución (previo al traslado a Colab Pro+).	32

	Resultados de la sonda local (10 %, fold 0, 1 época; prueba informal).	33
	Costo de cómputo.	33
	Decisiones.	33
	Limitaciones.	33
4.0.3	Task-20: Entorno canónico de ejecución en Google Colab Pro+ y sonda CUDA	34
	Objetivo.	34
	Procedimiento.	34
	Resultados (parciales).	34
	Interpretación.	35
	Decisiones.	35
	Limitaciones.	35
5	Síntesis de decisiones metodológicas	35

1. Propósito y convenciones

Este documento es el **registro empírico** de la ejecución del proyecto de grado. A diferencia del anteproyecto en `docs/proyecto_de_grado/`, que es prospectivo, la bitácora acumula evidencia conforme se completan las tareas del backlog. Cada hallazgo queda ligado a artefactos en `results/` y `models/`.

1.1 FLUJO DOCUMENTAL

El anteproyecto define el problema y el diseño metodológico. Esta bitácora registra lo ejecutado (tareas 3–16). El documento final (Trabajo de Grado – Frank Daza.tex, tarea 17) sintetiza los hallazgos con referencias cruzadas `\ref{hallazgo:task-N}`. Derivados posteriores: artículo científico (tarea 18) y ponencia (tarea 19).

1.2 TRAZABILIDAD OBLIGATORIA

Toda tarea del backlog que produzca evidencia experimental debe registrar un hallazgo con la etiqueta `\label{hallazgo:task-N}`, donde N es el identificador numérico de la tarea. Una tarea sin esta etiqueta se considera **no documentada**, aunque haya generado CSV en `results/`.

1.3 POLÍTICA DE BIBLIOGRAFÍA

La bibliografía es **compartida** con el anteproyecto: docs/proyecto_de_grado/Referencias.bib. Está prohibido duplicar entradas en un .bib local. Las citas nuevas se agregan con el skill agregar-cita sobre el archivo compartido. Motor: natbib + estilo apalike.

1.4 PLANTILLA ESTÁNDAR DE HALLAZGO

Toda tarea posterior debe replicar la siguiente estructura en el fragmento hallazgos/ que corresponda a su fase:

```
\subsubsection{Task-N: Título de la tarea}
\label{hallazgo:task-N}
\begin{tabularx}{\textwidth}{|l|X|}\hline
\textbf{Fecha} & YYYY-MM-DD \\\hline
\textbf{Actividad} & A# del anteproyecto \\\hline
\textbf{Artefactos} & \texttt{results/...}, \texttt{models/...} \\\hline
\end{tabularx}

\paragraph{Objetivo.} Qué se buscaba verificar.
\paragraph{Procedimiento.} Resumen reproducible (semilla, hiperparámetros, comando \texttt{uv run}).
\paragraph{Resultados.} Tablas y figuras con \texttt{\textbackslash input} desde \texttt{results/figures/}.
\paragraph{Interpretación.} Qué implica para la hipótesis de eficiencia de datos.
\paragraph{Decisiones.} Cambios congelados (p.\,ej.\ $L=4$) o riesgos abiertos.
\paragraph{Limitaciones.} Sesgos, supuestos, deuda técnica.
```

1.5 MAPEO DE FRAGMENTOS POR FASE

Fragmento	Tareas
h0_fundamentos.tex	1, 2, 3, 4, 5, 6
h1_arquitectura.tex	7, 8, 9, 10, 11
h2_experimentacion.tex	12, 13, 20
h3_analisis.tex	14, 15, 16

2. Fase 0 — Fundamentos reproducibles y datos

2.0.1 Task-1: Infraestructura y entorno reproducible

Fecha	2026-08-16
Actividad	Infraestructura (m-0)
Artefactos	src/config.py, src/utils/seed.py, src/utils/device.py, src/smoke.py, uv.lock, .python-version

Objetivo. Establecer un entorno reproducible con una única fuente de hiperparámetros, semilla global fijada y selección de dispositivo `cuda >mps >cpu`.

Procedimiento. Se implementó `ExperimentConfig` como `dataclass` congelada, `set_seed(42)` con `torch.use_deterministic_algorithms(True, warn_only=True)` y `get_device()` con la prelación definida en `AGENTS.md`. Verificación con:

```
uv run python -m src.smoke
```

Resultados. Stack congelado: Python 3.12, PyTorch 2.9.1, torchvision 0.24.1, `pennylane==0.45.0` (`pennylane-lightning==0.45.0` (0.45.1 de *lightning* no está publicado en PyPI), NumPy $\geq 2.0, < 2.3$. Dispositivo detectado: **mps**. Pérdida de referencia (dos corridas idénticas): **1.54205954**. Resultado reproducibilidad: **PASS**.

Interpretación. El entorno local cumple el criterio de reproducibilidad bit a bit dentro del mismo dispositivo. La semilla 42 queda fijada por convención del anteproyecto.

Decisiones. Semilla global 42 en `ExperimentConfig.semilla`. Gestor de paquetes exclusivo: UV (`uv sync --frozen`). Dispositivo registrado en cada corrida (contrato de tarea 4).

Limitaciones. La reproducibilidad bit a bit solo se garantiza dentro del mismo dispositivo; en `mps` algunas operaciones caen a CPU sin equivalente completo de `torch.cuda.manual_seed_all`.

2.0.2 Task-2: Estructura LaTeX del trabajo de grado y bitácora de hallazgos

Fecha	2026-08-16
Actividad	Documentación (m-0)
Artefactos	docs/trabajo_de_grado/ (bitácora, preámbulo, esqueleto de tesis, fragmentos hallazgos/)

Objetivo. Crear la estructura LaTeX del documento final y la bitácora metodológica de hallazgos, con trazabilidad `task-N` $\leftrightarrow$ artefactos en `results/`.

Procedimiento. Se copió el preámbulo UAO del anteproyecto a `preambulo_uao.tex` sin modificar el original. Se crearon los fragmentos `h0_fundamentos.tex` a `h3_analisis.tex`, el esqueleto Trabajo de Grado - Frank Daza.tex con capítulos vacíos y la cadena de compilación `pdflatex` $\rightarrow$ `bibtex` $\rightarrow$ `pdflatex` contra `../proyecto_de_grado/Referencias.bi`

Resultados. Estructura creada en `docs/trabajo_de_grado/`: `preambulo_uao.tex`, bitácora, esqueleto de tesis, carpetas `hallazgos/`, `capitulos/` y `Figuras/` (logo UAO). Bibliografía compartida sin entradas duplicadas. Cita de verificación: Penny-Lane [Bergholm et al. \(2018\)](#).

Interpretación. A partir de esta tarea, cada hallazgo experimental se registra en el fragmento de fase correspondiente. El documento final (tarea 17) sintetizará esta evidencia con `\ref{hallazgo:task-N}` en lugar de reconstruir resultados desde CSV.

Decisiones. Preámbulo copiado (no refactorizado) del anteproyecto entregado. Fragmentos sin espacios en el nombre de archivo. Logo UAO copiado (no symlink) para portabilidad entre máquinas.

Limitaciones. Los documentos principales (`Bitacora Metodologica de Hallazgos.tex`, `Trabajo de Grado - Frank Daza.tex`) llevan espacios en el nombre; la compilación requiere comillas en la línea de comandos.

2.0.3 Task-3: Auditoría del Brain Tumor MRI Dataset (A4)

Fecha	2026-08-17
Actividad	A4 — Curaduría y auditoría del dataset (m-0)
Artefactos	src/data/audit.py, results/dataset_manifest.csv, results/dataset_audit.md, results/figures/distribucion_clases.png

Objetivo. Verificar el conteo real, el balance por clase y partición, la integridad de las imágenes y la presencia de duplicados exactos antes de diseñar escenarios de escasez, siguiendo buenas prácticas de preparación de datos en imagen médica [Willemink et al. \(2020\)](#).

Procedimiento. Dataset local en notebooks/data/, enlazado por symlink a data/brain_tumor (ruta de ExperimentConfig). Recorrido estable con pathlib, verificación PIL (verify + load), hash SHA-256 de contenido y clasificación de duplicados en tres categorías. Ejecución:

```
uv run python -m src.data.audit
```

Resultados. Total inventariado: **7023** imágenes (coincide con el anteproyecto). Extensiones: solo .jpg. Imágenes corruptas: **0**. Duplicados exactos: **194** grupos con $n > 1$ (491 archivos involucrados); intra clase **194**, entre clases **0**, Training↔Testing **79** (subconjunto de los 194 grupos intra-clase). Exclusiones: **297** filas (duplicado_exacto: 194; fuga_train_test: 103). Utilizables: **6726**. La clase notumor concentra **269/297** exclusiones (90 %).

Tabla 1 — Inventario completo (7023 imágenes, antes de exclusiones):

Clase	Testing	Training	Total
glioma	300	1321	1621
meningioma	306	1339	1645
notumor	405	1595	2000
pituitary	300	1457	1757
Total	1311	5712	7023

Tabla 2 — Imágenes utilizables (6726, tras exclusiones):

Clase	Testing	Training	Total
glioma	299	1321	1620
meningioma	302	1333	1635
notumor	309	1422	1731
pituitary	295	1445	1740
Total	1205	5521	6726

Heterogeneidad: modos L (3093), RGB (3926), P/RGBA (4); resolución dominante 512×512 (4742 imágenes). Las cuatro clases corresponden a categorías del sistema nervioso central en la clasificación WHO 2021 [Louis et al. \(2021\)](#).

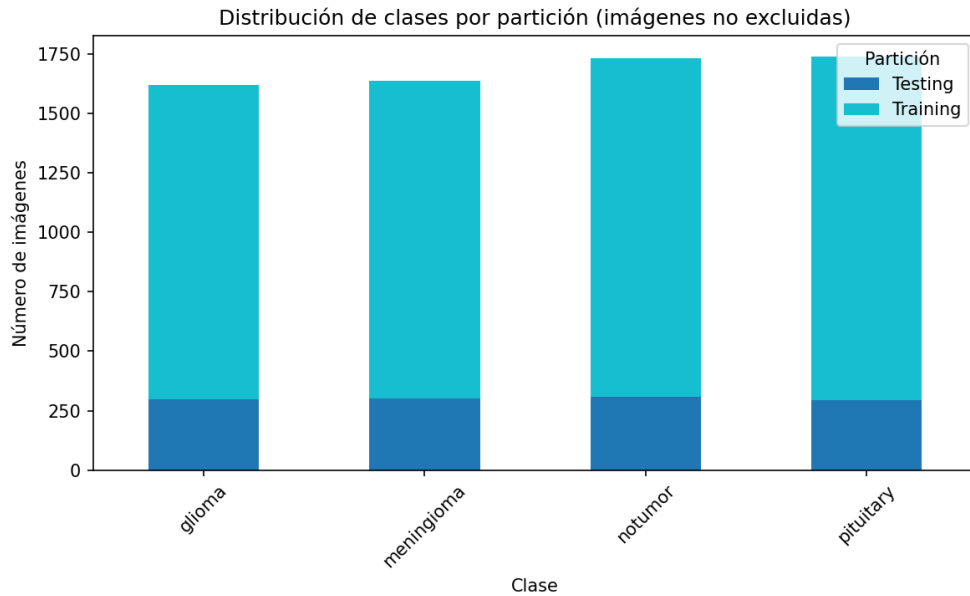


Figura 1: Distribución de clases por partición sobre las **6726** imágenes utilizables (no excluidas). La Tabla 1 del texto reporta el inventario completo de 7023.

Interpretación. El dataset agregado de Nickparvar [Nickparvar \(2023\)](#) es heterogéneo en modo de color y resolución; la auditoría confirma que el tamaño declarado es correcto pero existen duplicados exactos y fuga potencial entre Training/ y Testing/. Sin esta verificación, la exactitud de modelos posteriores podría inflarse por memorización de copias idénticas [Ait Haddou and Bennai \(2025\)](#). Tras la curaduría, el desbalance aparente entre notumor y las demás clases se reduce (2000 $\rightarrow$ 1731 frente a ~ 1620 –1740 en el resto).

Decisiones. Unir Training/ y Testing/ al conjunto completo para validación cruzada estratificada $k = 5$ (decisión D1, TASK-6). **Holdout externo considerado y**

descartado: el anteproyecto prescribe k -fold, no evaluación sobre el split original de Kaggle; además, 79 grupos de duplicados cruzan Training/ y Testing/, lo que invalidaría un holdout limpio. Las exclusiones quedan en `dataset_manifest.csv` mediante columnas `excluida` y `motivo_exclusion`; se conserva una copia por hash.

Limitaciones. El hash SHA-256 detecta duplicados byte a byte, no re-codificaciones JPEG con distinta compresión. La conversión a 3 canales y el redimensionado a 224×224 se documentan en TASK-5.

2.0.4 Task-4: Contrato de métricas, logging y artefactos

Fecha	2026-08-17
Actividad	Infraestructura (m-0) — contrato de resultados
Artefactos	<code>src/logging/records.py</code> , <code>src/logging/sinks.py</code> , <code>src/logging/timing.py</code> , <code>tests/test_records.py</code> , <code>tests/test_sinks.py</code>

Objetivo. Definir **antes de entrenar** el esquema único de resultados del proyecto: una fila por corrida (`modelo`, `data_fraction`, `fold`, `semilla`) con todas las métricas que exigen A9 y A10, más un historial por época en JSON para las curvas de A11. En problemas clínicos multiclase, la exactitud sola es insuficiente: deben reportarse sensibilidad y especificidad por clase [Esteva et al. \(2019\)](#).

Procedimiento. Se implementaron las dataclasses congeladas `RunRecord` y `EpochRecord` con validación explícita (`validar()`), la función `aplanar()` que alimenta CSV y wandb desde el mismo objeto, y los *sinks* de persistencia. El protocolo de tiempos exige descartar al menos 3 lotes de calentamiento, sincronizar el dispositivo (`cuda/mps`) antes y después de cada medición, y reportar la mediana de milisegundos por lote en inferencia. Verificación:

```
uv run pytest tests/ -q
```

Resultados. Esquema tidy de `results/experiments.csv` (una observación por fila):

Grupo	Columna	Descripción
Identidad	modelo	Identificador del modelo (hqcnn, efficientnet_b0, etc.)
Identidad	data_fraction	Fracción del entrenamiento usada
Identidad	fold	Índice del fold estratificado
Identidad	semilla	Semilla global de la corrida
Contexto	dispositivo	cuda, mps o cpu
Contexto	n_train, n_val	Tamaños de particiones
Contexto	epocas	Presupuesto de épocas
Contexto	n_params_entrenables	Parámetros con gradiente
Contexto	n_capas_vqc	Profundidad L del VQC (None en clásicos)
Contexto	commit_sha	SHA corto del commit (-dirty si aplica)
Contexto	timestamp	Marca temporal ISO 8601 de la corrida
Exactitud	accuracy_train, accuracy_val	Exactitud final
Pérdida	loss_train, loss_val	Pérdida media final
F1 (A9)	f1_val_weighted, f1_val_macro	F1 ponderado y macro
Por clase	sens_*, spec_*	Sensibilidad y especificidad por clase
Costo	train_time_s, inference_ms_per_batch	Tiempos medidos
Derivada	brecha_g	$ Acc_{train} - Acc_{val} $ (A11)

Historial por época en `results/history/{modelo}_{fraccion}_f{fold}_s{semilla}.json` con campos `epoca`, `loss_train`, `loss_val`, `accuracy_train`, `accuracy_val`. Pruebas automatizadas: **15 passed**.

Interpretación. El contrato convierte A9, A10 y A11 en consultas sobre datos ya recogidos. Si faltara un campo (p.ej. especificidad por clase), habría que repetir corridas cuyo gradiente se calcula por regla de cambio de parámetros. El Trainer (TASK-8) emitirá `RunRecord` sin conocer el formato de salida (DIP).

Decisiones. Orden canónico de clases: `glioma`, `meningioma`, `pituitary`, `notumor` (alineado con TASK-9). Un solo esquema vía `aplanar()` para CSV y wandb. Cabe-cera CSV validada contra `COLUMNAS_CSV`; escritura incompatible aborta en lugar de desalinear filas. Historial incluye `semilla` en el nombre para evitar colisiones futuras.

Limitaciones. El CSV en modo *append* no es seguro ante escritura paralela; si se paraleliza la campaña, escribir un CSV por corrida y consolidar en TASK-14. `wandb.init()` queda fuera del *sink*; el orquestador debe inicializar la sesión.

2.0.5 Task-5: Pipeline de preprocesamiento y aumento de datos (A5)

Fecha	2026-08-17
Actividad	A5 — Preprocesamiento y aumento de datos (m-0)
Artefactos	src/data/transforms.py, src/data/dataset.py, src/data/loaders.py, tests/test_transforms.py, tests/test_dataset.py, results/figures/ejemplos_aumento.png

Objetivo. Definir el pipeline de entrada para los *backbones* preentrenados en ImageNet: redimensionado a 224×224 , normalización con las estadísticas de ImageNet y aumento de datos **únicamente en entrenamiento**, siguiendo buenas prácticas de preparación de datos en imagen médica [Willemink et al. \(2020\)](#); [Litjens et al. \(2017\)](#).

Procedimiento. Transformaciones declaradas en `construir_transformaciones()` (API `torchvision.transforms.v2`, sin mezclar con la API clásica):

1. `Resize((224, 224))`
2. **Solo entrenamiento:** `RandomRotation( $\pm 10^\circ$ , fill=0)` y `RandomHorizontalFlip(p=0.5)`
3. `ToImage()` $\rightarrow$ `ToDtype(float32, scale=True)`
4. `Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225))`

El `MRIDataset` lee del manifiesto auditado (TASK-3), convierte siempre a RGB (incluidas imágenes en modo L) y mapea la etiqueta con `MAPEO_CLASE` alineado a TASK-4. Los `DataLoader` usan `generator` y `WorkerInitFn` (serializable en `spawn`) para reproducibilidad con `num_workers > 0`. Verificación:

```
uv run pytest tests/test_transforms.py tests/test_dataset.py -q
```

Resultados. Toda imagen sale como tensor `float32` de forma `(3, 224, 224)`. Pruebas automatizadas: **10 passed** (determinismo en validación, variación en entrenamiento, contrato de forma, grises a 3 canales, loader determinista con 2 workers).

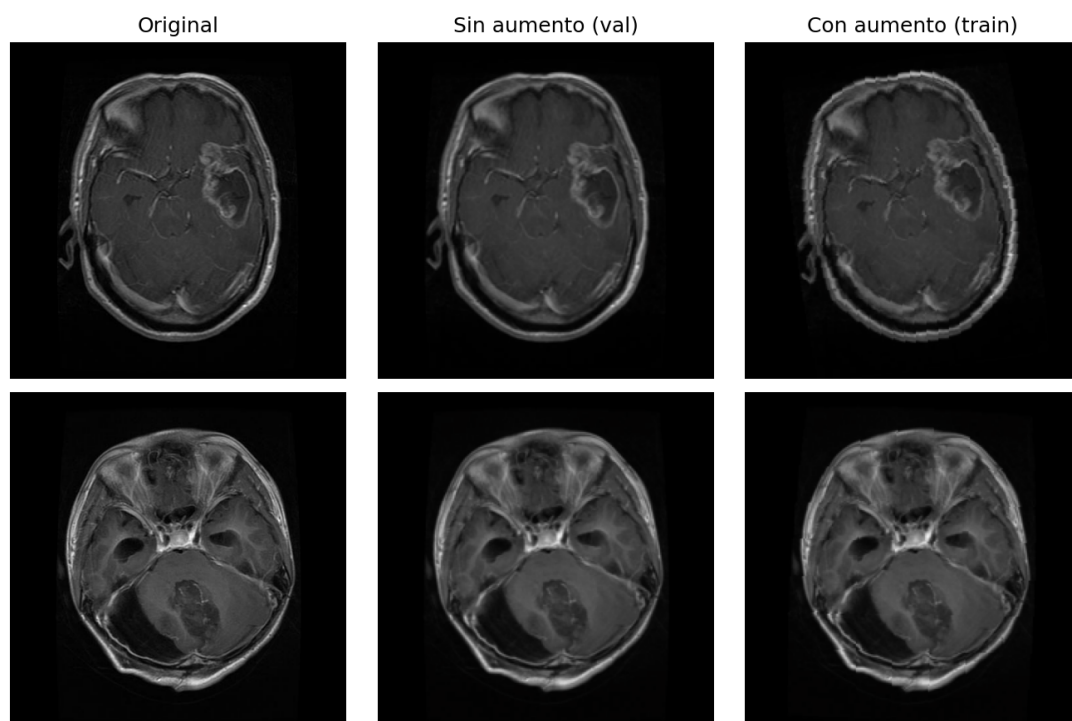


Figura 2: Ejemplos de preprocesamiento: original, validación (sin aumento) y entrenamiento (con rotación leve y volteo horizontal).

Interpretación. Los *backbones* EfficientNet-B0 y ResNet-50 esperan entradas calibradas con ImageNet [Tan and Le \(2019\)](#); normalizar con estadísticas del dataset propio degradaría las características transferidas de un modelo congelado. El aumento se aplica **después** de fijar las particiones (TASK-6): aumentar antes de partir filtraría variantes de la misma imagen entre entrenamiento y validación.

Decisiones. **Volteo horizontal** ($p = 0,5$): defendible por la simetría aproximada del cerebro en cortes axiales. **Rotaciones leves** ($\pm 10^\circ$): variabilidad de posicionamiento del paciente sin distorsionar la anatomía. **Volteo vertical descartado**: produce anatomías imposibles en MRI axial. $fill=0$ en rotación: coherente con el fondo negro típico de MRI. Validación y prueba **nunca** reciben aumento.

Limitaciones. El aumento no sustituye la auditoría de duplicados (TASK-3). La justificación anatómica no garantiza que toda variante sintética sea clínicamente plausible; un aumento agresivo introduciría sesgo en lugar de robustez [Willemink et al. \(2020\)](#).

2.0.6 Task-6: Protocolo de particiones — holdout, k-fold estratificado y fracciones (A7)

Fecha	2026-08-17
Actividad	A7 — Diseño de escenarios de escasez; base de A8 (m-0)
Artefactos	src/data/splits.py, results/splits.json, results/splits_summary.md, tests/test_splits.py

Objetivo. Fijar un protocolo único de particiones para que los tres modelos (HQCNN, EfficientNet-B0, ResNet-50) vean exactamente los mismos datos en cada celda del diseño factorial $3 \times 4 \times 5$, evitando que diferencias entre arquitecturas se confundan con ruido de remuestreo [Caro et al. \(2022\)](#); [Gupta et al. \(2025\)](#); [Khan et al. \(2024\)](#).

Procedimiento. Sobre el manifiesto auditado (6726 imágenes utilizables, TASK-3), ordenado por `ruta_relativa` con `mergesort`:

1. `StratifiedKFold` con $k = 5$, `shuffle=True`, `random_state=42` sobre el conjunto completo (decisión D1).
2. Validación de tamaño **fijo** por fold; idéntica en las cuatro fracciones.
3. Submuestreo estratificado **anidado** solo del entrenamiento en cascada 100 % $\rightarrow$ 50 % $\rightarrow$ 25 % $\rightarrow$ 10 % con `StratifiedShuffleSplit` y `train_size` entero.
4. Persistencia determinista en `results/splits.json` con hash SHA-256 del manifiesto y validación al cargar.

Ejecución:

```
uv run python -m src.data.splits
uv run pytest tests/test_splits.py -q
```

Decisión D1 frente a la redacción literal de A8. El anteproyecto prescribe validación cruzada estratificada en cada escenario de escasez. La lectura literal — submuestrear primero al 10 % y *luego* partir en folds— dejaría ~ 140 imágenes de validación por fold al 10 %, con varianza que dominaría el efecto buscado. Se adopta el orden inverso: **folds fijos primero; escasez solo en entrenamiento**. Así las

curvas de escasez miden *cantidad* de datos y no *identidad* de los datos (anidamiento garantizado). La validación fija también reduce la inestabilidad de estimadores por *split* único en imagen médica Singh et al. (2021). **Holdout externo descartado** (ver TASK-3): el split Kaggle no es defendible por duplicados cruzados entre Training/ y Testing/.

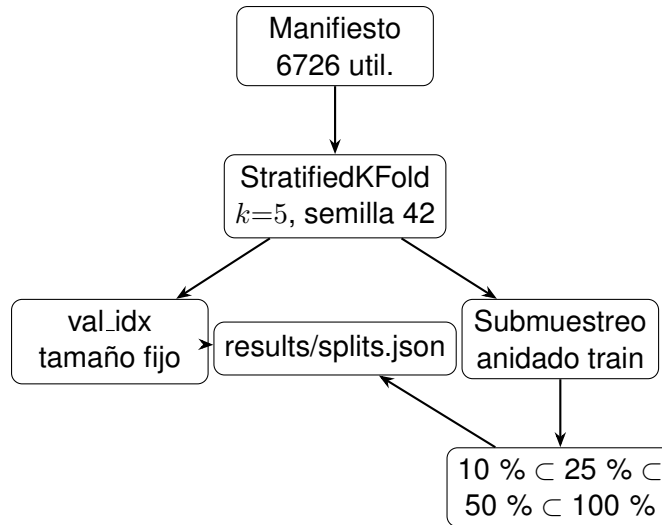


Figura 3: Diagrama del protocolo D1: folds fijos sobre el conjunto completo; escasez aplicada solo al entrenamiento con anidamiento entre fracciones.

Resultados. Artefacto: `results/splits.json` (5 folds $\times$ 4 fracciones). Hash del manifiesto: 30fb5b4334444b9d37924ac864d2cfc310347cdefb866ee23605214bbee129ca. Pruebas automatizadas: **9 passed** (anidamiento, estratificación $\pm 2\%$, disyunción train/val, validación idéntica entre fracciones, determinismo, hash).

Piso del escenario más extremo (10 % de entrenamiento, fold 0):

Fracción	Total train	glioma	meningioma	pituitary	notumor
0.10	538	130	131	139	138
0.25	1345	324	327	348	346
0.50	2690	648	654	696	692
1.00	5380	1296	1308	1392	1384

Validación fold 0: 1346 imágenes (constante en las cuatro fracciones). Folds 1–4: validación 1345 imágenes; entrenamiento al 100 %: 5381 imágenes. Mínimo por clase al 10 %: 130 imágenes ($\gg$ batch_size=32).

Interpretación. Persistir índices en disco convierte la comparación entre modelos en **pareada** y la campaña en reanudable: el Trainer (TASK-8) leerá `obtener_indices()` y nunca remuestreará. Sin anidamiento, una caída de exactitud al 10 % no distinguiría “menos datos” de “otros datos”.

Decisiones. Fracciones: 10 %, 25 %, 50 %, 100 %. Semilla global 42; semilla de submuestreo por fold: `semilla + k`. Tolerancia de estratificación declarada: $\pm 2\%$ respecto a la proporción global. `drop_last=True` en entrenamiento cuando el último lote sea de tamaño 1 (documentado para TASK-8).

Limitaciones. Los 5 folds **comparten** observaciones entre sí: no son 5 muestras independientes. Esta no independencia condiciona la ANOVA de TASK-15 y debe declararse en el análisis estadístico. Si el manifiesto cambia sin regenerar `splits.json`, los índices apuntan a otras imágenes sin error visible: por eso se valida el hash al cargar.

3. Fase 1 — Arquitectura híbrida CNN-VQC

3.0.1 Task-7: Extractor congelado y fábrica de backbones (A1)

Fecha	2026-08-17
Actividad	A1 — Extractor de características clásico preentrenado y congelado (m-1)
Artefactos	<code>src/models/backbones.py</code> , <code>src/models/heads.py</code> , <code>src/models/__init__.py</code> , <code>tests/test_backbones.py</code>

Objetivo. Construir una fábrica compartida de *backbones* EfficientNet-B0 y ResNet-50 con pesos ImageNet, con todas las capas convolucionales congeladas y una cabeza intercambiable que reduce el espacio latente a 4 dimensiones (número de clases y de qubits del VQC). Congelar el extractor hace viable el régimen de escasez: entrenar millones de parámetros con 10 % de los datos garantiza sobreajuste, mientras que entrenar solo la cabeza mantiene la capacidad proporcional a la evidencia [Mari et al. \(2020\)](#).

Procedimiento. `build_backbone(nombre)` carga pesos con la `Weights API` de `torchvision` (nunca `pretrained=True`): `EfficientNet-B0` con `IMAGENET1K_V1` [Tan and Le \(2019\)](#) y `ResNet-50` con `IMAGENET1K_V2` [He et al. \(2016\)](#). La cabeza de clasificación se sustituye por `nn.Identity()` para conservar el *pooling* adaptativo; todos los parámetros quedan con `requires_grad=False` y el módulo en `eval()`. La cabeza `CabeceraReduccion` implementa `Linear(dim_latente → 4)` y es compartida por la línea base clásica (TASK-12) y la reducción previa al VQC (TASK-10). Las versiones de pesos quedan registradas en `VERSIONES_PESOS` y `obtener_version_pesos()`. Verificación:

```
uv run pytest tests/test_backbones.py -q
```

Resultados. Dimensiones latentes y conteo de parámetros (calculado con `contar_parametros`).

Arquitectura	Versión pesos	Dim.	Congelados	Entrenables
EfficientNet-B0	IMAGENET1K_V1	1280	4 007 548	0 (backbone) / 5 124 (con cabeza)
ResNet-50	IMAGENET1K_V2	2048	23 508 032	0 (backbone) / 8 196 (con cabeza)

Pruebas automatizadas: **10 passed** (forma de salida, gradiente nulo en backbone, modo `eval()`, conteo de parámetros, filtro para optimizador, cabeza intercambiable).

Interpretación. Una única fábrica garantiza que HQCNN y líneas base comparten el mismo extractor y solo difieren en la cabeza posterior: es la condición para una comparación arquitectónica limpia sin ramas `if modelo == "hqcn"` en el bucle de entrenamiento (OCP). La reducción de 1280 o 2048 dimensiones a 4 es un **cuero de botella severo** impuesto por el número de qubits del VQC: no es una optimización de ancho de banda, sino una restricción del diseño; HQCNN y línea base compiten bajo la misma restricción, no en igualdad de información latente.

Decisiones. Versión de pesos ResNet-50: `IMAGENET1K_V2` (mejor exactitud que V1, receta de entrenamiento distinta; debe declararse al contrastar con la literatura). Sustituir la cabeza por `Identity` en lugar de recortar `features` evita errores de forma en el *pooling*. `parametros_entrenables()` filtra tensores para Adam (TASK-8): pasar `modelo.parameters()` completo mantiene momentos espurios en parámetros congelados.

Limitaciones. `requires_grad=False` no impide que BatchNorm actualice estadísticas móviles si el módulo está en `train()`. `build_backbone()` devuelve `eval()`; mantener ese modo cuando el Trainer llama `model.train()` es responsabilidad del módulo contenedor híbrido (TASK-10). La normalización de entrada debe ser la de ImageNet (TASK-5); un backbone congelado con estadísticas distintas desplaza las características transferidas.

3.0.2 Task-8: Bucle unificado de entrenamiento y evaluación (Trainer)

Fecha	2026-08-17
Actividad	Bucle único de entrenamiento y evaluación para líneas base, ablación y campaña (m-1)
Artefactos	<code>src/train/trainer.py</code> , <code>src/train/dataloading.py</code> , <code>src/train/__init__.py</code> , <code>tests/test_trainer.py</code>

Objetivo. Implementar un único bucle de entrenamiento y evaluación que reciba un `nn.Module` ya construido y un `ExperimentConfig`, de modo que HQCNN y líneas base clásicas compitan bajo el mismo protocolo experimental [Esteva et al. \(2019\)](#); [Litjens et al. \(2017\)](#). Evitar bucles distintos por familia de modelo: basta una diferencia en optimizador o en medición de tiempos para invalidar la comparación arquitectónica.

Procedimiento. La clase `Trainer` depende solo de `nn.Module` y `ExperimentConfig` (DIP): no importa `pennylane` ni `torchvision`. El helper `construir_loaders_para_fold()` lee índices de `results/splits.json` vía `obtener_indices()` sin remuestreo; el cargador de entrenamiento usa `drop_last=True`. Hiperparámetros congelados en `ExperimentConfig`: 15 épocas, Adam con `lr=1e-3`, `CrossEntropyLoss` sobre *logits* (no probabilidades). Criterio de época reportada: **presupuesto fijo completo** (última época), sin seleccionar la mejor época mirando el fold de validación que luego se reporta (evita fuga por selección de época). Emite `RunRecord` y `EpochRecord`; reanudabilidad por tupla (`modelo`, `data_fraction`, `fold`, `semilla`); pesos en `models/{modelo}_{frac}_f{fold}_s{semilla}` con `state_dict()` exclusivamente. Tiempos de inferencia con calentamiento y sincronización de dispositivo (TASK-4). Durante el entrenamiento, el Trainer muestra una barra `tqdm` por lote (una línea reescrita en consola) y un `logger.info` por época con pérdidas, exactitud de validación, tiempo por época y ETA; sin logs por muestra ni por paso de `backward`. Las barras se desactivan si no hay TTY (pytest). Verificación:

```
uv run pytest tests/test_trainer.py -q
```

```
uv run pytest tests/ -q
```

Resultados. Hiperparámetros comunes congelados antes de cualquier campaña:

Parámetro	Valor congelado
Épocas	15
Optimizador	Adam
Tasa de aprendizaje	10^{-3}
Función de pérdida	CrossEntropyLoss (sobre logits)
Criterio de época reportada	Última época del presupuesto fijo
Clave de reanudabilidad	(modelo, data_fraction, fold, semilla)

Pruebas automatizadas: **8 passed** en `test_trainer.py`; regresión completa **95 passed**. Sustituibilidad verificada: el mismo `Trainer` entrena un modelo clásico trivial y uno con capa cuántica simulada (logits en $[-1, 1]$) sin ramas condicionales por tipo de modelo.

Interpretación. Un solo bucle convierte la comparación HQCNN vs. línea base en una sustitución de modelo (LSP): la diferencia observada es atribuible a la arquitectura, no a protocolos distintos. El criterio de época fija elimina el sesgo optimista de elegir la mejor validación sobre el mismo fold reportado. `n_capas_vqc` se obtiene por *duck typing* (`getattr(modelo, "n_capas_vqc", None)`), sin ramas `if modelo == "hqcn"`.

Decisiones. Separar `construir_loaders_para_fold()` del `Trainer` preserva SRP: el bucle entrena y evalúa; no arma particiones. Los *sinks* (CSV, wandb, JSON) quedan fuera del `Trainer`: este emite registros y el orquestador (TASK-13) persiste. Solo parámetros con `requires_grad=True` entran a Adam. Presupuesto de 15 épocas es el de diseño; queda sujeto a confirmación tras la sonda de 1 época en Colab/CUDA (TASK-13, decisión D2).

Limitaciones. El `Trainer` llama `model.train()` en cada época; mantener el backbone congelado en `eval()` es responsabilidad del módulo contenedor híbrido (TASK-10). Los logits del VQC en $[-1, 1]$ achatan el *softmax* y limitan la confianza alcanzable; se retoma en TASK-10. La campaña factorial (60 celdas) requiere orquestador externo (TASK-13); este entregable no incluye `src/train.py`.

3.0.3 Task-9: Clasificador cuántico variacional de 4 qubits (A2)

Fecha	2026-08-17
Actividad	A2 — Diseño del clasificador cuántico variacional (m-1)
Artefactos	src/models/vqc.py, src/models/__init__.py, tests/test_vqc.py, Figuras/circuito_vqc.png

Objetivo. Diseñar el circuito variacional de 4 qubits que clasifica las cuatro clases de MRI mediante mediciones locales, con codificación por ángulos y ansatz de entrelazamiento fuerte. Cada decisión de arquitectura debe tener respaldo bibliográfico explícito: la codificación define el mapa de características cuántico [Schuld and Killoran \(2019\)](#); con 4 características y 4 qubits el circuito permanece superficial, requisito práctico en la era NISQ [Preskill \(2018\)](#); el ansatz *Strongly Entangling Layers* ofrece expresividad con $O(L \cdot n)$ parámetros [Schuld et al. \(2020\)](#).

Procedimiento. El QNode `circuito_vqc` declara `interface="torch"` y `diff_method="parameter_jacobian"` sobre `default.qubit`. La codificación usa `AngleEmbedding` con `rotation="Y"`; el ansatz es `StronglyEntanglingLayers` con profundidad L tomada de `ExperimentConfig.n_capas`. Las mediciones son locales: `qml.expval(qml.Z(i))` por qubit, en lugar de un observable global, porque los costos globales inducen mesetas áridas incluso a profundidad reducida, mientras que los observables locales preservan gradientes medibles [Cerezo et al. \(2021\)](#). El mapeo qubit→clase se deriva de `CLASES_ORDEN` en una constante única (`MAPEO_QUBIT_CLASE`). Los pesos se inicializan cerca de la identidad con escala 0,01 para mitigar mesetas áridas de inicialización aleatoria [McClean et al. \(2018\)](#); [Grant et al. \(2019\)](#). La forma de los pesos se obtiene de `StronglyEntanglingLayers.shared_weights` vía `forma_pesos_vqc()`. El diagrama se genera con `qml.draw_mpl`. Verificación:

```
uv run pytest tests/test_vqc.py -q
uv run python src/models/vqc.py
```

Resultados. Mapeo qubit→clase (idéntico al Dataset):

Qubit	Clase
0	glioma
1	meningioma
2	pituitary
3	notumor

Norma media del gradiente en la inicialización (20 muestras, semilla 42), diagnóstico previo a la ablación de TASK-11:

L	$\ \nabla\theta\ $ inicial
2	1.237
4	1.730
6	2.107

En este rango de profundidad la norma *aumenta* con L , lo que indica que la inicialización cercana a la identidad mantiene gradientes medibles; la selección final de L queda para TASK-11, donde expresividad y entrenabilidad se miden en el dataset real [Holmes et al. \(2022\)](#).

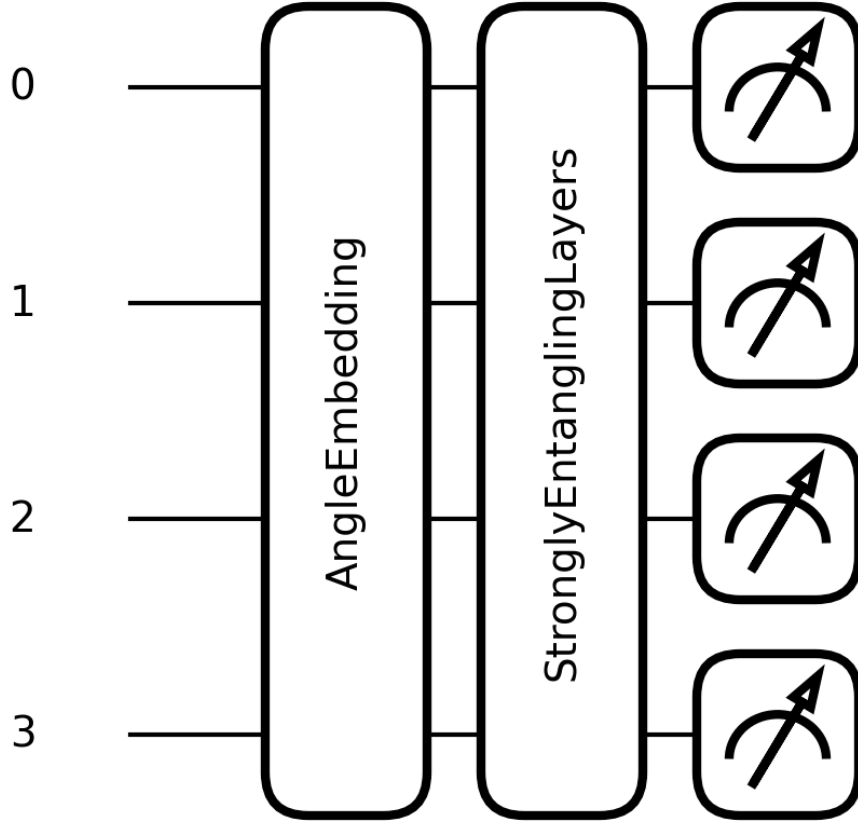


Figura 4: Circuito VQC de 4 qubits con AngleEmbedding (Y), ansatz StronglyEntanglingLayers ($L = 4$) y mediciones locales $\langle Z_i \rangle$.

Pruebas automatizadas: **7 passed** en `test_vqc.py`; regresión completa **64 passed**.

Interpretación. Un observable por qubit da un mapeo natural de 4 qubits a 4 clases sin costo global que induzca mesetas áridas [Cerezo et al. \(2021\)](#). Con `parameter-shift` y $L = 6$ hay $6 \times 4 \times 3 = 72$ parámetros, es decir 144 evaluaciones de circuito por muestra y por paso de optimización; esta aritmética justifica la compuerta de presupuesto de TASK-11. Las salidas $\langle Z_i \rangle \in [-1, 1]$ por construcción; la capa densa de TASK-10 debe acotar las entradas antes del *embedding* para evitar el *wrapping*

periódico de los ángulos.

Decisiones. `rotation="Y"` en `AngleEmbedding`; `default.qubit` con `parameter-shift` (no `lightning.qubit` como `default`). Escala de inicialización 0,01. API vigente: `qml.Z(i)`, no `qml.PauliZ(i)`. No se cita `anzatz_vqc_2024` como respaldo del `ansatz`.

Limitaciones. Las entradas deben acotarse antes del *embedding* (TASK-10 con `tanh` escalado). La selección de L no se fija por intuición sino con la ablación de TASK-11. El diagnóstico de gradiente usa entradas aleatorias en $[0, \pi]$; el rango real dependerá de la capa densa acotada.

3.0.4 Task-10: Modelo híbrido HQCNN con TorchLayer (A3)

Fecha	2026-08-17
Actividad	A3 — Integración del modelo híbrido HQCNN (m-1)
Artefactos	<code>src/models/hqcnn.py</code> , <code>src/models/__init__.py</code> , <code>tests/test_hqcnn.py</code>

Objetivo. Integrar en un único `nn.Module` el flujo de extremo a extremo: backbone EfficientNet-B0 congelado (TASK-7) → reducción lineal → acotación de ángulos → VQC vía `qml.qnn.TorchLayer` (TASK-9) → 4 logits para `CrossEntropyLoss`. Es el punto donde el diseño deja de ser dos piezas y pasa a ser un modelo entrenable con un solo grafo de diferenciación [Bergholm et al. \(2018\)](#); [Mitarai et al. \(2018\)](#). El patrón de transferencia clásico-cuántico con extractor congelado sigue a [Mari et al. \(2020\)](#).

Procedimiento. La clase HQCNN reutiliza `build_backbone(.efficientnet_b0)` y `CabeceraReduccion(1280 → 4)` (misma cabeza que la línea base clásica de TASK-12). El `QNode circuito_vqc` se envuelve con `TorchLayer`, declarando `set_input_argument(.ent` porque PennyLane exige un argumento de datos con nombre fijo. Los pesos variables se inicializan con `inicializar_pesos` vía `init_method` (escala 0,01, TASK-9), no con el uniforme $[0, 2\pi]$ por defecto de `TorchLayer`.

La salida de la reducción se acota con $\tanh(x) \cdot \pi$ antes del `AngleEmbedding`: los ángulos son periódicos módulo 2π y, sin acotación, dos características distintas pueden codificarse en el mismo estado sin error visible.

HQCNN.train() mantiene el backbone en eval() para que BatchNorm no actualice estadísticas móviles. El extractor corre bajo torch.no_grad(); el gradiente fluye solo hacia reduccion y vqc.

Con parameter-shift, PennyLane no admite retropropagación en lotes sobre el QNode (issue #4462): el forward itera muestra a muestra sobre TorchLayer y apila los logits.

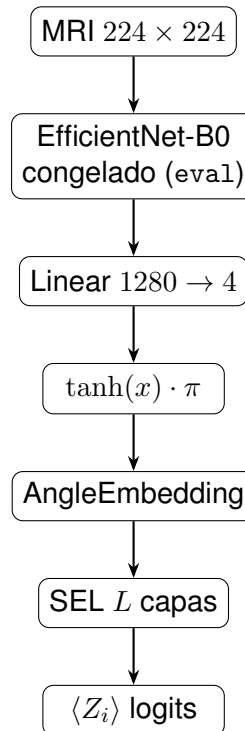


Figura 5: Flujo HQCNN de extremo a extremo (TASK-10): extractor congelado, reducción, acotación de ángulos, VQC vía TorchLayer y 4 logits.

Verificación:

```
uv run pytest tests/test_hqcnn.py -q
```

Resultados. Conteo de parámetros por bloque (contar_parametros_por_bloque(), $L = 4$):

Bloque	Parámetros	Entrenable
Backbone EfficientNet-B0	4 007 548	No (congelado)
Reducción CabeceraReduccion	5 124	Sí
VQC StronglyEntanglingLayers	48	Sí
Total entrenable	5 172	

Verificación numérica del gradiente (peso de mayor magnitud, CPU, float32, $\varepsilon = 10^{-4}$, semilla 42): analítico = $8,18 \times 10^{-2}$, numérico = $8,17 \times 10^{-2}$ (error relativo observado $< 0,2$ % en esa corrida; la prueba automatizada usa `rtol=0.15` por inestabilidad de `parameter-shift` en float32).

Los logits son $\langle Z_i \rangle \in [-1, 1]$ por construcción; la confianza máxima del *softmax* con 4 clases queda acotada. Con logits $(1, -1, -1, -1)$:

$$p_{\text{máx}} = \frac{e^1}{e^1 + 3e^{-1}} \approx 0,711.$$

Es propiedad del diseño, no un error de entrenamiento; no se añade factor de escala aprendible para preservar la comparabilidad factorial (TASK-11/13).

Pruebas automatizadas: **12 passed** en `test_hqcnn.py`; el mismo `Trainer` de TASK-8 entrena el HQCNN una época sin ramas condicionales.

Interpretación. El HQCNN es sustituible por una línea base clásica en el bucle de entrenamiento (LSP): solo cambia el módulo posterior al extractor congelado. Con 5 172 parámetros entrenables frente a ~ 4 millones congelados, el modelo híbrido concentra la capacidad de adaptación en la cabeza y el circuito variacional, coherente con el régimen de escasez [Mari et al. \(2020\)](#). La iteración por muestra en el VQC es el cuello de botella de cómputo que TASK-11 debe cuantificar antes de la campaña factorial.

Decisiones. `CabeceraReduccion` en lugar de `nn.Linear` suelto (paridad con `baseline`). `init_method` con `inicializar_pesos`. Sin `softmax` antes de `CrossEntropyLoss`. Persistencia exclusiva con `state_dict()`. Propiedad `n_capas_vqc` para duck typing en `RunRecord`.

Limitaciones. La iteración por muestra escala linealmente con el tamaño de lote y con `parameter-shift` (2 evaluaciones por parámetro). El backbone solo soporta EfficientNet-B0 en esta entrega; ResNet-50 queda para líneas base (TASK-12). La

verificación numérica del gradiente usa el peso de mayor magnitud porque el primero puede tener gradiente casi nulo en la inicialización.

3.0.5 Task-11: Ablación de profundidad L y presupuesto de cómputo (A2)

Fecha	2026-08-17
Actividad	A2 — Selección de profundidad del ansatz y compuerta de viabilidad (m-1)
Artefactos	src/experiments/ablacion_L.py, results/ablacion_L.csv, results/selected_hparams.json, results/figures/ablacion_L_curvas_perdi results/history/hqcnn_0p25_f0_s42_L{2,4,6}.json

Objetivo. Elegir $L \in \{2, 4, 6\}$ con un protocolo económico y, en la misma corrida, medir el costo real de la campaña factorial de TASK-13. Congelar L en `selected_hparams.json` y registrar una decisión *go/no-go* sobre la viabilidad del diseño [Holmes et al. \(2022\)](#); [McClellan et al. \(2018\)](#); [Caro et al. \(2022\)](#).

Protocolo (decisión de diseño, no A8). Un solo fold (0), fracción del 25 %, presupuesto de 5 épocas (un tercio de las 15 de campaña), semilla 42, Trainer de TASK-8 sin bucle ad hoc, simulador `default.qubit` con `parameter-shift`. **No** se reporta como resultado con significancia estadística: es una decisión instrumentada antes de la campaña factorial.

Criterio de selección (pre-declarado). Mayor F1 macro de validación al final del presupuesto reducido; ante diferencias $\leq 0,02$, preferir la L más baja por costo y entrenabilidad.

Procedimiento.

```
uv run python -m src.experiments.ablacion_L
uv run pytest tests/test_ablacion_L.py -q
```

TorchLayer con `parameter-shift` no es estable en mps; la ablación se ejecutó en cpu para tiempos extrapolables con el simulador `default.qubit`.

Resultados. Tabla comparativa (fold 0, 25 %, 5 épocas, $n_{\text{train}} = 1345$, dispositivo cpu):

L	F1 macro	Acc. val	Loss val	s/época	n_q
2	0.594	0.672	0.867	314	24
4	0.659	0.734	0.790	544	48
6	0.817	0.810	0.878	842	72

Normas de gradiente inicial: $L = 2$: 1.237; $L = 4$: 1.730; $L = 6$: 2.107 (aumentan con L ; sin evidencia de meseta árida en la inicialización cercana a la identidad).

Interpretación CE vs. F1. $L = 6$ maximiza F1 macro (0.817) pero *CrossEntropyLoss* de validación (0.878) supera a $L = 4$ (0.790): con logits en $[-1, 1]$ la pérdida no es monótona con la calidad de clasificación. Por eso el criterio pre-declarado usa F1 macro, no loss.

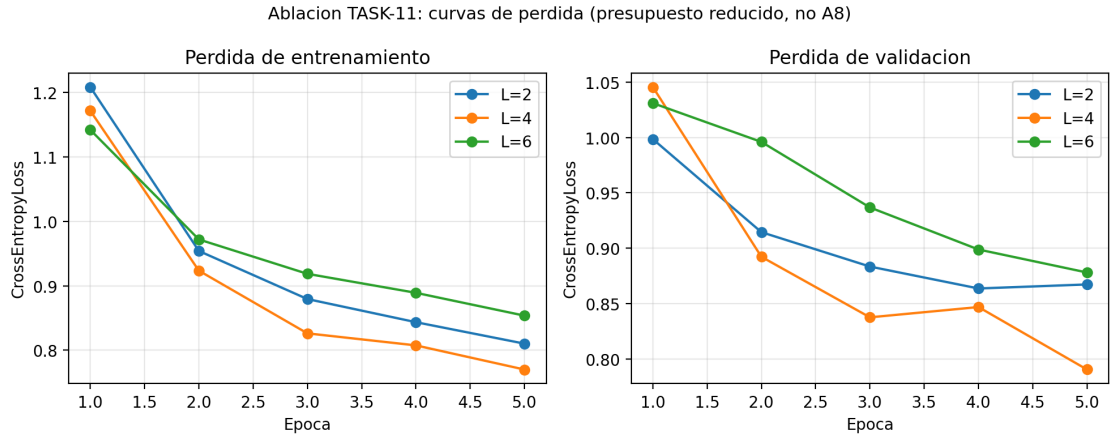


Figura 6: Curvas de pérdida train/val por profundidad en el presupuesto reducido de TASK-11. Permiten distinguir arquitectura peor de no convergencia en 5 épocas.

Presupuesto extrapolado. Con $L = 6$ (842 s/época al 25 %, 15 épocas de campaña): HQCNN solo ≈ 129.8 h en cpu; cota conservadora (3 modelos $\times$ 4 fracciones $\times$ 5 folds) ≈ 389.4 h $>$ umbral 72 h $\Rightarrow$ *no-go* local. No se recortan épocas sin evidencia. *Go* condicionado a sonda de 1 época en `colab_cuda` antes de TASK-13; D2 (HQCNN al 100 %) se confirma tras re-extrapolar. Con 4 qubits y `default.qubit`, el VQC no se acelera sustancialmente en GPU; Colab aporta sesión larga y GPU para baselines y backbone.

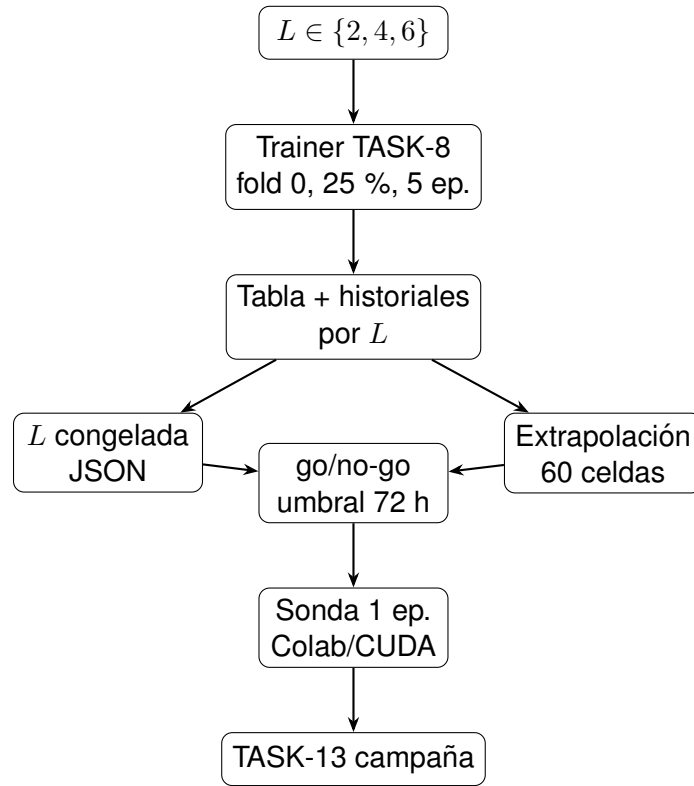


Figura 7: Compuerta de viabilidad D2: ablación económica, extrapolación y *go* condicionado a re-medicación en Colab antes de la campaña factorial.

Decisiones.

- **L congelada:** 6 (results/selected_hparams.json, commit 5153d2c-dirty).
- **Compuerta:** *no-go* en CPU local; *go* condicionado a sonda Colab/CUDA. mitigacion_adoptado null (sin recorte de épocas).
- Mitigaciones evaluadas sin adopción silenciosa: precálculo de características (incompatible con augmentation, TASK-5), reducir épocas, reducir k , restringir diseño factorial (rompe ANOVA, TASK-15).
- **Historiales:** results/history/hqcn_0p25_f0_s42_L2.json, L4.json, L6.json (sufijo $L\{n\}$ evita sobrescritura).

Limitaciones. Un fold y 5 épocas no permiten inferencia estadística. La extrapolación en cpu no predice tiempos en Colab/CUDA. La cota conservadora asume que las baselines cuestan como el HQCNN (las clásicas son más rápidas).

4. Fases 2 y 3 — Experimentación y análisis

4.0.1 Task-12: Líneas base clásicas con el 100 % del conjunto (A6)

Fecha	2026-08-17
Actividad	A6 — Entrenamiento de las líneas base clásicas (m-2)
Artefactos	src/models/baseline.py, src/models/factory.py, src/experiments/baselines.py, tests/test_baseline.py, results/experiments.csv, results/baselines_summary.csv, models/*.pt, results/history/*.json

Objetivo. Establecer el límite superior de exactitud contra el cual se interpretará el HQCNN y el resto de la campaña factorial (TASK-13). Las líneas base EfficientNet-B0 y ResNet-50 comparten el mismo `Trainer` (TASK-8), las mismas particiones de `results/splits.json` (TASK-6) y el mismo presupuesto de 15 épocas, con extractor ImageNet congelado y cabeza `Linear(dim → 4)` (TASK-7).

Procedimiento. `ClassicalBaseline` envuelve `build_backbone + CabeceraReduccion` y mantiene el backbone en `eval()` durante el entrenamiento. La fábrica `build_model(nombre, cfg)` unifica la construcción para baselines y HQCNN (sin ramas `if` en el orquestador). Versiones de pesos: EfficientNet-B0 IMAGENET1K_V1 [Tan and Le \(2019\)](#); ResNet-50 IMAGENET1K_V2 [He et al. \(2016\)](#) (registradas en `VERSIONES_PESOS`). Dispositivo de ejecución: `mps` (macOS). Antes de cada corrida, verificar `indices_fold` comprueba que los índices cargados coinciden con `splits.json`. Ejecución:

```
uv run python -m src.experiments.baselines
uv run pytest tests/test_baseline.py -q
```

Resultados por fold (validación, fracción 1.00).

Modelo	Fold	Acc. val	F1 macro	F1 pond.	Tiempo (s)
EfficientNet-B0	0	0.9376	0.9368	0.9378	640
EfficientNet-B0	1	0.9390	0.9377	0.9387	643
EfficientNet-B0	2	0.9346	0.9335	0.9347	645
EfficientNet-B0	3	0.9368	0.9360	0.9371	663
EfficientNet-B0	4	0.9368	0.9356	0.9368	782
ResNet-50	0	0.9190	0.9168	0.9181	1028
ResNet-50	1	0.9152	0.9133	0.9148	982
ResNet-50	2	0.9257	0.9240	0.9252	987
ResNet-50	3	0.9182	0.9164	0.9177	1012
ResNet-50	4	0.9234	0.9218	0.9231	942

Agregación sobre 5 folds (media $\pm$ desv. estándar).

Métrica	EfficientNet-B0	ResNet-50
Exactitud val.	0,9369 $\pm$ 0,0012	0,9203 $\pm$ 0,0039
F1 macro	0,9358 $\pm$ 0,0012	0,9185 $\pm$ 0,0040
F1 ponderado	0,9370 $\pm$ 0,0011	0,9198 $\pm$ 0,0039
Brecha G	0,0273 $\pm$ 0,0021	0,0229 $\pm$ 0,0037
Inferencia (ms/lote)	52,7 $\pm$ 14,8	140,9 $\pm$ 73,0

Pruebas automatizadas: **19 passed** (test_baseline.py + test_backbones.py). Diez celdas en experiments.csv con clave (modelo, 1.00, fold, semilla=42).

Contraste con la literatura. [Ait Haddou and Bennai \(2025\)](#) reportan 96.48 % de precisión con HQCM-EBTC sobre el mismo *dataset*, entrenando una CNN personalizada desde cero con el conjunto completo y el *split* original Training/Testing. [Khan et al. \(2024\)](#) evalúan un HQCNN híbrido en condiciones distintas (arquitectura, preprocesamiento y partición). **Los números no son directamente comparables** con los de esta tarea porque: (i) validación cruzada estratificada de 5 folds frente a un único *holdout*; (ii) extractor preentrenado congelado frente a entrenamiento completo del backbone; (iii) cuello de botella a 4 dimensiones (restricción del diseño híbrido, ver Hallazgo 3.0.1). La exactitud observada ($\sim$ 93.7 % EfficientNet-B0, $\sim$ 92.0 % ResNet-50) no indica fallo de implementación: es coherente con la restricción de entrenar solo la cabeza lineal sobre características fijas.

Interpretación. EfficientNet-B0 supera a ResNet-50 bajo la misma restricción de extractor congelado y la misma cabeza, con menor varianza entre folds (desv. std. de exactitud 0.12 vs. 0.39 puntos porcentuales). La brecha de generalización G se

mantiene baja ($\sim 2-3$ puntos), coherente con pocos parámetros entrenables. Estas 10 celdas quedan disponibles para reutilización por TASK-13 sin reentrenamiento (*corrida.completada*).

Decisiones. ResNet-50 con IMAGENET1K_V2 (no intercambiable con V1 en comparaciones bibliográficas). Dispositivo `mps` para baselines clásicas (a diferencia del HQCNN, que usa CPU por inestabilidad de `TorchLayer` en MPS, Hallazgo 3.0.5).

Limitaciones. El extractor congelado limita el techo alcanzable: la tabla **no** representa el máximo de EfficientNet-B0 o ResNet-50 con *fine-tuning* completo. Los 5 folds comparten observaciones (TASK-6); la desviación estándar entre folds mide estabilidad de la partición, no error de muestreo independiente. El tiempo de inferencia de ResNet-50 es mayor por la mayor dimensión latente (2048 vs. 1280) y el costo del *forward* congelado.

Addenda (decisión D3): reejecución en CUDA. Las diez corridas de esta tabla se midieron en `mps`. Al trasladarse la campaña de TASK-13 íntegramente a Colab Pro+ con `cuda` (Hallazgo 4.0.3), la reutilización anunciada en el párrafo anterior queda **revocada**: mantener diez celdas en `mps` dentro de un diseño de sesenta celdas en `cuda` confundiría el dispositivo con el efecto del modelo en el tiempo de entrenamiento y la latencia de inferencia que exige A9 (TASK-14), justo en la fracción del 100 % donde se contrasta el HQCNN contra las líneas base. Las filas y los historiales se conservan como evidencia en `results/historico_mps.csv` y `results/history_mps/`, y las celdas se repiten en GPU dentro de TASK-13. El protocolo de A6 no cambia: se repite la medición, no el diseño. Los valores de exactitud de esta tabla quedan además como contraste de sanidad para las corridas en CUDA.

4.0.2 Task-13: Campaña experimental k-fold en escenarios de escasez (A8)

Fecha	2026-08-17
Actividad	A8 — Campaña factorial con validación cruzada (m-2)
Artefactos	<code>src/experiments/campana.py</code> , <code>tests/test_campana.py</code> , <code>results/experiments.csv</code> , <code>results/campana_estado.json</code> , <code>results/history/*.json</code> , <code>models/*.pt</code>

Objetivo. Ejecutar el diseño factorial balanceado $\{\text{HQCNN, EfficientNet-B0, ResNet-50}\} \times \{10\%, 25\%, 50\%, 100\%\} \times 5 \text{ folds} = 60 \text{ celdas}$, todas ejecutadas en el mismo disposi-

tivo (cuda) dentro del entorno de Colab Pro+ que entrega TASK-20 (Hallazgo 4.0.3). La comparación es **pareada**: todas las celdas comparten `results/splits.json` y el mismo presupuesto de 15 épocas, condición necesaria para estimar la interacción *modelo* $\times$ *fracción* en TASK-15 Caro et al. (2022); Gupta et al. (2025).

Diseño e implementación. El orquestador `src/experiments/campana.py` delega el entrenamiento al Trainer (TASK-8), construye modelos vía `build_model` (TASK-12) y congela $L = 6$ desde `results/selected_hparams.json` (TASK-11). Dispositivo: cuda para las sesenta celdas. La prelación local (baselines en mps, HQCNN en cpu por inestabilidad de TorchLayer + parameter-shift en MPS, Hallazgo 3.0.5) deja de aplicar en Colab, donde `get_device` y `get_device_hqcnn` convergen en cuda. La reanudabilidad compara la clave completa (modelo, fracción, fold, semilla). El estado de ejecución se persiste en `results/campana_estado.json`; las celdas fallidas se registran allí con motivo, no en el CSV oficial. Ejecución por bloques, de menor a mayor costo y con el HQCNN al 100 % al final:

```
python -m src.experiments.campana --fraccion 1.00 --modelo efficientnet_b0
python -m src.experiments.campana --fraccion 1.00 --modelo resnet50
python -m src.experiments.campana --fraccion 0.10
python -m src.experiments.campana --fraccion 0.25
python -m src.experiments.campana --fraccion 0.50
python -m src.experiments.campana --fraccion 1.00
python -m src.experiments.campana --verificar
```

Los dos primeros bloques rehacen en GPU las líneas base al 100 % archivadas de TASK-12 y permiten contrastar la exactitud contra los valores conocidos en mps antes de comprometer horas en el bloque híbrido.

Estado de ejecución (previo al traslado a Colab Pro+).

Estado	Celdas	Notas
Completadas	0	—
Omitidas	0	—
Pendientes	60	Campaña completa en cuda
Fallidas	0	—
Archivadas (TASK-12)	10	Baselines al 100 % en mps, se repiten en GPU
Archivadas (sonda)	3	Prueba informal de 1 época, fuera del CSV oficial

Esta tabla se actualiza **al cerrar cada bloque** `--fraccion`, no al final de la campaña, para que la bitácora refleje el avance real y el costo acumulado.

Resultados de la sonda local (10 %, fold 0, 1 época; prueba informal).

Modelo	Acc. val	Tiempo (s)	Dispositivo
EfficientNet-B0	0.7192	33.3	mps
ResNet-50	0.6322	35.4	mps
HQCNN ($L = 6$)	0.3908	388.9	cpu

Pruebas automatizadas: **10 passed** (`test_campana.py`). Estas tres corridas validan el flujo de persistencia, no el diseño experimental: con una sola época carecen de validez inferencial y se archivan en `results/pruebas_informales.csv` (TASK-20). Dejarlas en el CSV oficial habría hecho que `Trainer.corrida_completada()` las omitiera por reanudabilidad, mezclando corridas de 1 época con corridas de 15 en la ANOVA de TASK-15.

Costo de cómputo. La sonda local midió el HQCNN en CPU a $\sim 12\times$ el tiempo por época de EfficientNet-B0 en MPS, lo que confirma que el bloque híbrido domina el presupuesto. La estimación vigente de TASK-11 (389.41 h para 60 celdas, extrapolación conservadora en CPU) se sustituye por la re-extrapolación en `cuda` de TASK-20; el costo real acumulado se compara contra ambas con `comparar_costo` y se registra aquí al cerrar cada bloque.

Decisiones. Runtime único `cuda` en Colab Pro+ y homogeneidad de hardware en las sesenta celdas (decisión D3, Hallazgo 4.0.3). Orden de ejecución: líneas base al 100 % primero como revalidación en GPU, luego fracciones crecientes, y HQCNN al 100 % al cierre. Validación cruzada estratificada de 5 folds: la varianza entre folds mide estabilidad de la partición, no error de muestreo independiente Singh et al. (2021) (limitación declarada desde TASK-6).

Limitaciones. La campaña no está ejecutada: las 60 celdas quedan pendientes tras el traslado a Colab Pro+. Los AC de integridad (60 filas sin duplicados, historial completo, `dispositivo=cuda`) se verificarán con `--verificar` al terminar. La reejecución en GPU de las diez celdas al 100 % de TASK-12 implica que sus métricas de exactitud pueden diferir levemente de las registradas en `mps` por no determinismo entre dispositivos; la comparación entre ambas sirve como control de sanidad, no como réplica bit a bit.

4.0.3 Task-20: Entorno canónico de ejecución en Google Colab Pro+ y sonda CUDA

Fecha	2026-08-17 (en curso)
Actividad	Soporte a A8; recurso declarado en el anteproyecto (Colab Pro+)
Artefactos	notebooks/colab_campana.ipynb, results/historico_mps.csv, results/history_mps/, results/pruebas_informales.csv, results/selected_hparams.json, .cursor/skills/ejecutar-experimento/SKI

Objetivo. Habilitar el único entorno donde la campaña de TASK-13 es viable y cerrar la compuerta de presupuesto que TASK-11 dejó abierta. Tres condiciones deben cumplirse antes de entrenar: (i) medir una época del HQCNN en `cuda` para sustituir la extrapolación en CPU; (ii) garantizar que las sesenta celdas del diseño factorial compartan dispositivo; (iii) sacar del CSV oficial toda corrida que no pertenezca al protocolo, sin destruirla.

Procedimiento. Cuaderno delgado que importa `src.experiments.campana` y no reimplementa modelos, particiones ni bucles de entrenamiento. El contrato de rutas hacia Google Drive vive únicamente en el cuaderno: `src/` sigue derivando todas sus rutas de `ExperimentConfig` con `pathlib`, como exige TASK-1. El conjunto de datos se copia comprimido al disco local del runtime antes de descomprimirse (leer directamente desde Drive domina el tiempo por época); `results/` y `models/` se enlazan a Drive para que un corte de sesión no destruya el CSV, los historiales ni los pesos. Dependencias con los pines del README (PyTorch 2.9.1 + cu128, PennyLane 0.45.1), única excepción documentada a UV. `wandb` opera con `WANDB_MODE=offline` y se sincroniza después, para no perder corridas cuando la sesión se interrumpe. Antes de cualquier entrenamiento se archivan las diez filas en `mps` y las tres de la sonda local, y se reponen esas celdas como pendiente en `results/campana_estado.json`. GPU prevista: T4.

Resultados (parciales). Higiene del registro ejecutada: `archivar_corridas_no_cuda` retiró 10 corridas heterogéneas (líneas base al 100 % en `mps`) y 3 pruebas informales (sonda de 1 época), movió sus 13 historiales a `results/history_mps/` y dejó `experiments.csv` con cero filas y las 60 celdas en pendiente. Los historiales de la ablación de L (TASK-11) permanecen en `results/history/` por no pertenecer al CSV de la campaña. La CLI expone `--archivar-no-cuda` y `--sin-baselines-previas`; esta última permite arrancar sin las líneas base en el CSV sin relajar la validación de L ni el hash de `splits.json`. Suite de pruebas: **118 passed**, con cuatro casos nuevos que cubren el archivado, la reposición de celdas a pendiente y la idempotencia.

Pendiente de ejecución en Colab. Al completarse se registran aquí: verificación de `torch.cuda.is_available()` y modelo de GPU asignada, segundos por época del HQCNN ($L = 6$) al 10 % en `cuda`, factor de aceleración frente a los 388.9 s medidos en CPU, horas re-extrapoladas para las 60 celdas y la decisión de presupuesto resultante en `results/selected_hparams.json`.

Interpretación. El cuello de botella del proyecto no es la CNN sino la simulación del VQC con `parameter-shift` sobre `default.qubit`, que se evalúa en CPU aunque el resto del grafo esté en GPU. Por eso la elección de acelerador se toma por costo-beneficio y no por potencia nominal: una GPU de gama alta acelera el *backbone* congelado y las líneas base, pero no el término dominante del costo. La sonda es lo que permite decidir con una medición en lugar de una expectativa.

Decisiones. Decisión D3 de la síntesis metodológica: Colab Pro+ con `cuda` como runtime único de entrenamiento; las sesenta celdas en el mismo dispositivo; corridas en `mps` y sonda local archivadas, no eliminadas. El análisis posterior (TASK-14, TASK-15, TASK-16) se ejecuta en CPU para no consumir unidades de cómputo. No se recortan épocas, folds ni celdas: las mitigaciones evaluadas y rechazadas en TASK-11 siguen rechazadas.

Limitaciones. Colab no garantiza el mismo modelo de GPU entre sesiones; si la asignación cambia a mitad de campaña, el dispositivo sigue siendo `cuda` pero los tiempos por época dejan de ser estrictamente comparables entre bloques, y ese cambio debe anotarse en el estado de ejecución de TASK-13. La reproducibilidad bit a bit solo se garantiza dentro del mismo dispositivo (TASK-1), de modo que las corridas en GPU no replican exactamente las métricas medidas en `mps`. El entorno no es reproducible sin acceso a la cuenta de Drive del autor: lo que sí queda reproducible es el código, los pines de versión y los artefactos versionados en `results/`.

5. Síntesis de decisiones metodológicas

Esta sección consolida las decisiones congeladas a medida que avanza el proyecto:

- **D1** — Orden de particiones y fracciones de escasez (tarea 6). Ver [2.0.6](#) y Figura 3.

- Unir Training/ y Testing/ tras auditoría (6726 imágenes utilizables).
 - StratifiedKFold con $k = 5$, random_state=42, sobre el conjunto completo.
 - Validación de tamaño fijo por fold; idéntica en las cuatro fracciones de escasez.
 - Submuestreo estratificado **anidado** solo del entrenamiento: $10 \% \subset 25 \% \subset 50 \% \subset 100 \%$.
 - Índices persistidos en results/splits.json con validación de hash del manifiesto.
 - Holdout externo sobre Testing/ descartado (duplicados cruzados y diseño del anteproyecto).
 - Lectura literal de A8 (submuestrear antes de partir) rechazada: varianza de validación dominaría al 10 %.
- **D2** — Profundidad L del VQC y presupuesto de cómputo (tarea 11). Ver [3.0.5](#) y Figura [7](#).
- Ablación económica: $L \in \{2, 4, 6\}$, fold 0, fracción 25 %, 5 épocas (no A8).
 - Criterio pre-declarado: mayor F1 macro; empate $\leq 0,02$ favorece L menor.
 - L congelada en results/selected_hparams.json (valor y artefactos en TASK-11).
 - Extrapolación: horas HQCNN solo vs. cota conservadora (3 modelos); umbral 72 h.
 - No-go en CPU local si supera umbral; go condicionado a sonda de 1 época en Colab/CUDA (TASK-13) antes de las 60 celdas.
 - Sin recorte de épocas sin evidencia de convergencia; D2 (HQCNN al 100 %) se confirma tras re-extrapolar.
 - Historiales por L en results/history/ con sufijo $_L\{n\}$.
- **D3** — Entorno de ejecución de la campaña y homogeneidad de hardware (tareas 20 y 13). Ver [4.0.3](#).
- Runtime único de entrenamiento: Google Colab Pro+ con GPU cuda, recurso ya declarado en el anteproyecto (Tecnologías Duras). El equipo local queda reservado para análisis y redacción.
 - Las 60 celdas del diseño factorial se ejecutan en el mismo dispositivo: ninguna celda que entre al análisis proviene de mps ni de cpu.
 - Las 10 corridas al 100 % de la tarea 12, medidas en mps, se archivan en results/historico_mps.csv (historiales en results/history_mps/) y se reejecutan en CUDA.

- Motivo: A9 exige reportar tiempo de entrenamiento y latencia de inferencia. Con hardware mezclado esas métricas quedarían confundidas con el dispositivo justo en la fracción del 100 %, que es donde se contrasta el HQCNN contra las líneas base.
- La sonda local de 1 época se archiva en `results/pruebas_informales.csv`: sale del CSV oficial sin borrarse, para que la reanudabilidad no la confunda con una celda válida.
- Costo de la homogeneidad: $\sim 1.5\text{--}3$ h de GPU, frente a las ~ 130 h del bloque HQCNN.
- La compuerta abierta en D2 (*go* condicionado a sonda en `colab_cuda`) se cierra con la re-extrapolación registrada en `results/selected_hparams.json`.

Referencias

- Ait Haddou, M. and Bennai, M. (2025). Hqcm-ebtc: A hybrid quantum-classical model for explainable brain tumor classification. *arXiv preprint arXiv:2506.21937*.
- Bergholm, V., Izaac, J., Schuld, M., Gogolin, C., Alam, M. S., Ahmed, S., et al. (2018). PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*.
- Caro, M. C., Huang, H.-Y., Cerezo, M., Sharma, K., Sornborger, A., Cincio, L., and Coles, P. J. (2022). Generalization in quantum machine learning from few training data. *Nature Communications*, 13(1):4919.
- Cerezo, M., Sone, A., Volkoff, T., Cincio, L., and Coles, P. J. (2021). Cost function dependent barren plateaus in shallow quantum neural networks. *Nature Communications*, 12(1):1791.
- Esteva, A., Chou, K., Yeung, S., Naik, N., Madani, A., Mottaghi, R., Kasstein, Y., Tan, R., Nguyen, T., Maggio, R., et al. (2019). A guide to deep learning in healthcare. *Nature medicine*, 25(1):24–29.
- Grant, E., Wossnig, L., Ostaszewski, M., and Benedetti, M. (2019). An initialization strategy for addressing barren plateaus in parametrized quantum circuits. *Quantum*, 3:214.
- Gupta, R. S., Wood, C. E., Engstrom, T., Pole, J. D., and Shrapnel, S. (2025). Quantum machine learning for digital health? a systematic review. *npj Digital Medicine*, 8:237.

- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778.
- Holmes, Z., Sharma, K., Cerezo, M., and Coles, P. J. (2022). Connecting ansatz expressibility to gradient magnitudes and barren plateaus. *PRX Quantum*, 3(1):010313.
- Khan, M. A.-Z., Galib, A. A. O., Innan, N., and Bennai, M. (2024). Brain tumor diagnosis using hybrid quantum convolutional neural networks. *arXiv preprint arXiv:2401.15804*. Preprint disponible en arXiv.
- Litjens, G., Kooi, T., Bejnordi, B. E., Setio, A. A. A., Ciompi, F., Ghafoorian, M., van der Laak, J. A., van Ginneken, B., and Sánchez, C. I. (2017). A survey on deep learning in medical image analysis. *Medical image analysis*, 42:60–88.
- Louis, D. N., Perry, A., Wesseling, P., Brat, D. J., Cree, I. A., Figarella-Branger, D., Hawkins, C., Ng, H., Pfister, S. M., Reifenberger, G., et al. (2021). The 2021 who classification of tumors of the central nervous system: a summary. *Neuro-oncology*, 23(8):1231–1251.
- Mari, A., Bromley, T. R., Izaac, J., Schuld, M., and Killoran, N. (2020). Transfer learning in hybrid classical-quantum neural networks. *Quantum*, 4:340.
- McClean, J. R., Boixo, S., Smelyanskiy, V. N., Babbush, R., and Neven, H. (2018). Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9(1):4812.
- Mitarai, K., Negoro, M., Kitagawa, M., and Fujii, K. (2018). Quantum circuit learning. *Physical Review A*, 98(3):032309.
- Nickparvar, M. (2023). Brain tumor mri dataset (version 3).
- Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, 2:79.
- Schuld, M., Bocharov, A., Svore, K. M., and Wiebe, N. (2020). Circuit-centric quantum classifiers. *Physical Review A*, 101(3):032308.
- Schuld, M. and Killoran, N. (2019). Quantum machine learning in feature hilbert spaces. *Physical Review Letters*, 122(4):040504.
- Singh, V., Pencina, M. J., Einstein, A. J., Liang, J. X., Berman, D. S., and Slomka, P. J. (2021). Impact of train/test sample regimen on performance estimate stability of machine learning in cardiovascular imaging. *Scientific Reports*, 11(1):14490.
- Tan, M. and Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR.

Willemink, M. J., Koszek, W. A., Hardell, C., Wu, J., Fleischmann, D., Harvey, H., Folio, L. R., Summers, R. M., Rubin, D. L., and Lungren, M. P. (2020). Preparing medical imaging data for machine learning. *Radiology*, 295(1):4–15.